

Seminar 6: CURSORS

- Declaration
- Open and position the pointer on the first record
- Load the current line
- Close the cursor

In Oracle, the cursor is a pointer (identifier) towards the memory called context.

The implicit cursor is automatically created and managed by the system for a number of commands (*create, alter, insert, delete* etc.).

The explicit cursor is automatically created and managed by the programmer, especially for Select statements.

The cursor mechanism is the RDBMS solution for passing from dealing with data as a set (table) to dealing with data at record level. The cursor is an area of internal memory in which a set of tuples is brought and from which a record can be extracted for processing. The cursor is based on a retrieval that is stored in the DB dictionary.

The attributes for the implicit cursor are:

- **SQL%ROWCOUNT**
- **SQL%FOUND**
- **SQL%NOTFOUND**

Examples:

Delete a row from Product_information table and count the number of rows deleted.

DECLARE

no **NUMBER**(2);

BEGIN

DELETE FROM product_information

WHERE product_id=3001;

no:=SQL%ROWCOUNT;

DBMS_OUTPUT.PUT_LINE (no || ' rows deleted');

COMMIT;

END;

Try to modify the name of the product 113. If the product doesn't exist (the UPDATE command doesn't modify anything), display a message.

BEGIN

UPDATE product_information

SET product_name='PC'

WHERE product_id=113;

IF SQL%NOTFOUND THEN

DBMS_OUTPUT.PUT_LINE('The product 113 doesn't exist');

END IF;

END;

Delete a customer whose id is given by the user through a substitution variable.

```
ACCEPT var_id PROMPT 'Give an id for the customer:'
```

```
VARIABLE no varchar2(100)
```

```
DECLARE
```

```
BEGIN
```

```
DELETE FROM customers WHERE customer_id=&var_id; --211
```

```
:no:=TO_CHAR(SQL%ROWCOUNT)||' deleted rows';
```

```
END;
```

```
/
```

```
PRINT no
```

```
Rollback;
```

EXPLICIT CURSOR

- It is used for individually processing each line (record) that is loaded in the memory by a SELECT statement.
- The set of rows returned by the SELECT query is called *result set*.
- The cursor keeps a pointer to the current line in a result set.

Verifying the state of an explicit cursor is made using the following attributes:

- **cursor_name%ISOPEN** - returns TRUE if the cursor is opened;
- **cursor_name%NOTFOUND** - returns TRUE if the most recent FETCH didn't return any row;
- **cursor_name%FOUND** – the complement of %NOTFOUND;
- **cursor_name%ROWCOUNT** – the number of rows returned up until the current moment.

Processing the explicit cursor assumes following these steps:

- 1) declare the variables in which the values of a cursor's row will be loaded;
- 2) declare the explicit cursor, by giving it a name and defining the query that needs to be processed by it;

```
DECLARE CURSOR cursor_name IS SELECT.....;
```

- 3) open the cursor by using the *OPEN* statement, which executes the query. The returned rows are *the active set of data*, which can now be loaded.

```
OPEN cursor_name;
```

- 4) by using the *FETCH* statement, data from the current row of the cursor is loaded in variables. Each loading makes the pointer move to the next line of the active set of data.

```
FETCH cursor_name INTO var1, var2,.....;
```

- 5) close the cursor with *CLOSE* statement, which disposes the active set of data. The cursor can be later opened in order to obtain a new active set of data.

```
CLOSE cursor_name;
```

Usually, in order to process the rows of an explicit cursor a loop is defined, which executes a *FETCH* in each iteration. In the end, all the rows in the active data set are

processed and an unsuccessfully FETCH makes the %NOTFOUND attribute return TRUE.

Before the first FETCH, %NOTFOUND returns NULL. The same thing happens when FETCH never successfully executes.

Example: Display the regions that begin with A or a.

```
DECLARE
  cursor cc is select region_id, region_name from regions where
    upper(region_name) like 'A%';
  id regions.region_id%type;
  name regions.region_name%type;
BEGIN
  dbms_output.put_line('The regions that begin with A:');
  open cc;
  loop
    fetch cc into id, name;
    exit when cc%notfound;
    dbms_output.put_line('The region '||id||' is called: '||name);
  end loop;
  close cc;
end;
```

Example: Display the most expensive 3 products.

```
DECLARE
  cursor cc is select * from product_information where list_price is not null order by
    list_price desc;
  product cc%rowtype;
BEGIN
  dbms_output.put_line('The most expensive products are:');
  open cc;
  loop
    fetch cc into product;
    exit when cc%notfound or cc%rowcount=4;
    dbms_output.put_line('The product '||product.product_id||' has the price:
    '||product.list_price);
  end loop;
  close cc;
end;
```

Implicitly managing the cursor by using FOR loops

FOR record_name IN cursor_name LOOP

 ...
END LOOP;

The record type doesn't have to be declared (as any counter used in a FOR structure). For the cursor the operations open, fetch and close are made implicitly.

Exemplu: Display the regions that begin with A or a.

DECLARE

cursor cc is select region_id, region_name from regions where upper(region_name) like 'A%';

BEGIN

dbms_output.put_line('The regions that begin with A:');

for region in cc loop

dbms_output.put_line('The region '||region.region_id||' is called: '||region.region_name);

end loop;

end;

Using the cursor directly in a FOR structure

In this case, the cursor isn't declared, doesn't have a name and cannot be managed. The disadvantage is that its attributes cannot be used. The only thing that can be used is the processing of data made by the SELECT statement.

FOR record_name IN (SELECT.....) LOOP

...

END LOOP;

Example: Display the total quantity bought for each product.

begin

dbms_output.put_line('Total quantity for each product:');

for rec in

(select p.product_id prod, sum(quantity) total_quantity

from product_information p, order_items o

where p.product_id=o.product_id

group by p.product_id)

loop

dbms_output.put_line(rec.total_quantity||' pieces of product '||rec.prod||' were ordered');

end loop;

end;

Using the cursor with parameters

For a higher flexibility, cursors with parameters can be used. They send the values of the actual parameters in SQL queries. The declaration of such a cursor is made like this:

Cursor cursor_name (parameter1 data_type, ...)

Is select ...;

Open the cursor:

Open cursor_name(parameter1_value,.....);

The parameter cursors don't have an additional functionality, they just represent a clear and simple way of specifying the input values. The type of these parameters is scalar, but their precision is not mentioned.

Example: Display the products that have a total ordered quantity larger than a value received as parameter.

DECLARE

CURSOR c_prod (p_val NUMBER) IS

select p.product_id prod, sum(quantity) total_quantity

from product_information p, order_items o

where p.product_id=o.product_id

group by p.product_id

HAVING sum(quantity)>p_val

ORDER BY total_quantity desc;

v_val NUMBER(5);

rec_prod c_prod%rowtype;

BEGIN

v_val:=50;

DBMS_OUTPUT.PUT_LINE('The products that have a total ordered quantity larger than '|| v_val);

IF NOT c_prod%ISOPEN THEN

OPEN c_prod (v_val);

END IF;

LOOP

FETCH c_prod into rec_prod;

EXIT WHEN c_prod%notfound;

DBMS_OUTPUT.PUT_LINE(rec_prod.total_quantity||' pieces of product '||rec_prod.prod||' were ordered');

END LOOP;

CLOSE c_prod;

END;

OPTIONAL

FOR UPDATE clause

- FOR UPDATE clause is added in the query associated with the cursor in order to block the affected rows when the cursor is opened.

CURSOR C IS SELECT FROM....

FOR UPDATE [OF COLUMN_NAME] [NOWAIT|WAIT n];

- NOWAIT - causes an exception if the rows are locked by another session;
- WAIT n – if necessary, waits n seconds for the rows to be unlocked. If after the n seconds the rows are still blocked, it will trigger an exception.

Example: For Orders table update the order_date by adding a week to each date. Use a FOR UPDATE cursor to block the rows in the table when the cursor is opened.

```
DECLARE
CURSOR c_update IS
SELECT *
FROM orders
FOR UPDATE OF order_date NOWAIT;
BEGIN
FOR rec IN c_update LOOP
DBMS_OUTPUT.PUT_LINE('The order '||rec.order_id||' has the date:
'||rec.order_date);
UPDATE orders
SET order_date=order_date+7
WHERE order_id=rec.order_id;
DBMS_OUTPUT.PUT_LINE('The order '||rec.order_id||' has the date:
'||rec.order_date);
END LOOP;
END;
/
```

```
SELECT * FROM orders;
```

Notice: In the previous example, it can be seen that the cursor doesn't update in the same time with the table. It has to be closed and then opened again in order to see the updates.

In order to better manipulate Update and Delete commands **WHERE CURRENT OF** clause can be used. It allows modifying the records based on the current line of the cursor. FOR UPDATE clause has to be used when defining the cursor in order to block the rows when executing the Open statement.

UPDATE table

SET field=....

WHERE CURRENT OF cursor_name;

Example: For Orders table update the order_date by decreasing it by one week. Use FOR UPDATE and WHERE CURRENT OF clauses.

```
DECLARE
CURSOR c_update IS
SELECT *
FROM orders
FOR UPDATE OF order_date NOWAIT;
BEGIN
FOR rec IN c_update LOOP
DBMS_OUTPUT.PUT_LINE('The order '||rec.order_id||' has the date:
'||rec.order_date);
UPDATE orders
SET order_date=order_date-7
WHERE CURRENT OF c_update;
```

```
DBMS_OUTPUT.PUT_LINE('The order '||rec.order_id||' has the date:  
'||rec.order_date);  
END LOOP;  
END;  
/
```

```
SELECT * FROM orders;
```

Homework:

1. Display all the customers and their orders. Use a cursor to load the customer's name and another cursor with parameter to load their orders.
2. Display the orders that have the largest value. Stop after the first 5 orders.